Paper Summary

Introduction

The increasing complexity of products such as spacecraft has intensified demands on system design and verification, driving adoption of Model-Based Systems Engineering (MBSE). MBSE aims to support all lifecycle phases through unified, formalized models, transitioning from document-based to model-driven R&D. However, mainstream MBSE methods face challenges in consistently addressing both system architectures and physical properties, as different characterization methods lead to poor model consistency. Approaches such as metamodel transformation and co-simulation require multiple heterogeneous languages and tools, increasing the learning curve for engineers and restricting the application of intelligent automation across multi-level models. Unifying the modeling and simulation process is therefore a key trend, with efforts such as SysML 2.0, the KARMA specification, and the authors' own X language — an integrated modeling and simulation language based on DEVS extensions — providing foundations for intelligent generative design.

Large-scale Generative AI (GenAI), primarily in the form of large language models (LLMs) such as GPT-4, Code-Llama, WizardCoder, Code-Qwen, and CodeGeeX, has demonstrated strong capabilities in code generation and information extraction, making its integration into MBSE increasingly feasible. While prior work has explored using LLMs for requirements standardization, UML diagram generation, and modeling task assistance, most GenAI applications in MBSE focus on requirement and functional levels, with limited research on generating simulation models for the system physical properties of products. This paper therefore proposes an innovative generative system design framework for MBSE that employs BERT-based inference, Transformer-based generative models, and X language to construct simulation models for system physical properties from product design documents, introducing scalable simulation model templates, tailored evaluation metrics, and a validation and simulation framework to improve model quality and accelerate product design.

X language simulation model template construction

Basic concepts of DEVS. DEVS is a formal modeling specification for discrete event systems built around two core constructs. The Atomic Model captures system behavior and state through a seven-tuple $\langle S, ta, \delta_{int}, X, \delta_{ext}, Y, \lambda \rangle$, compris-

ing a state set, input/output event sets, internal and external transition functions, an output function, and a time advance function. The Coupled Model integrates multiple atomic or coupled models via a four-tuple $\langle D, EIC, EOC, IC \rangle$ that defines component sets and external/internal coupling relationships. DEVS operates on an event-driven, discrete-time principle; while extensions such as Hybrid DEVS support continuous–discrete hybrid systems, current tools (e.g., CD++ and DEVS-JAVA) offer limited and non-standardized hybrid modeling support.

Basic concepts of X language. X language is a modeling and simulation language developed on top of XDEVS, itself an extension of DEVS that adds continuous port connectivity to support hybrid models. Unlike standard languages such as SysML, X language supports integrated modeling of both system architecture and physical characteristics, and provides dual-mode (graphical and textual) representations. Its three core classes are: the **Couple class**, corresponding to the DEVS coupled model, which uses the keywords `Part` and `Connection` to describe system composition and connectivity; the **Discrete class**, an atomic model for discrete-event systems using keywords `State` and `Transform` to represent state transitions; and the **Continuous class**, an atomic model for continuous systems using the keyword `Equation` to describe dynamic behavior via non-causal, equation-based modeling. All X language class elements map directly to their DEVS counterparts, with additional extensions beyond the base formalism.

X language class template construction. To address the challenges of generating accurate and consistent simulation code for complex, multi-scale products, the paper introduces scalable templates for each X language class. These templates decompose model syntax while preserving key structural elements, allowing model length and content to be dynamically adjusted based on the number and attributes of included elements. Code generation is reframed as a modular code completion task, which mitigates performance degradation, context loss, and output truncation caused by excessively long inputs. Template construction follows three rules: (1) a top-down design principle, constructing system-level templates before component-level ones; (2) extraction of the most effective structural descriptions to minimize LLM-generated text and improve accuracy; and (3) explicit clarification of inter-module coupling relationships. Concrete scalable templates are defined for the Couple class, Discrete class, and Continuous class, with placeholder syntax (e.g., `<DiscreteName>`, `<Equation>`) enabling flexible instantiation during generation.

Simulation model generation method based on scalable templates and transformer-based models

The authors present a three-step simulation model generation framework that integrates BERT-based inference, Transformer-based generative models, and X language scalable templates to automatically construct simulation models for system physical properties from product design documents. In **Step 1**, a fine-tuned BERT model performs Named Entity Recognition (NER) and single-sentence classification on the product design document to filter irrelevant content and build a simulation model corpus. In **Step 2**, NER-BERT identifies system composition and containment relationships, while the classification BERT extracts connectivity-related sentences; together these outputs are used to populate the couple class template (keywords `Import`, `Part`, and `Connection`) and generate the top-level couple class model via a Transformer-based model guided by tailored prompts. In **Step 3**, atomic class models are derived by extracting subsystem ports from the couple class `Connection` keyword via Algorithm 1, and the behavioral components — keyword `State` for discrete class models and keyword `Equation` for continuous class models — are generated through a Few-shot prompted, fine-tuned Transformer-based model (CodeQwen1.5-7B, fine-tuned using LoRA via LLaMA Factory). Any undefined functions discovered in the atomic class code trigger additional generation of X language function class models.

The training data for the Transformer-based model is augmented using GPT-3.5 to address the scarcity and homogenization of the existing X language model library. Prompts for data generation consist of three components — Introduction (X language BNF and few-shot examples), Input (model description), and Task — and the resulting synthetic models are combined with the original library to form the training corpus. A mask-based code completion format is used: the `Value` and `State` (or `Equation`) keywords are masked, and the model is trained to predict them from the remaining code. The optimization objective follows standard conditional language model training (maximizing $\max_{\Phi} \sum_{(x,y)\in Z} \sum_{t=1}^{|y|} \log P_{\Phi}(y_t \mid x, y_{<t})$), and LoRA decomposition ($h = W_0 x + BAx$, with $r \ll \min(d, k)$) reduces computational overhead.

The framework also introduces a dedicated evaluation methodology for the generated simulation models. Rather than relying solely on standard code metrics such as CodeBLEU or Pass@k, the authors define two MBSE-specific metrics: **Degree of Error** (quantifying the actual impact of errors and the effort required for correction, accounting for both syntax and simulation logic errors via attenuation

coefficients α_i and β_i) and **Model Consistency** (verifying name consistency between parent and subsystem models, port consistency between the couple class `Connection` and atomic class `Port`, and function behavioral consistency). Correctness similarity scores for couple class and atomic class models are computed via weighted combinations (Equations 8 and 11), and the entropy weight method (EWM) is used to objectively determine component and model weights. A final composite score $\text{Score} = A_{\text{parent}} \cdot \sum_{i=1}^{n} C_{\text{subsystem},i} \cdot A_{\text{subsystem},i}$ aggregates performance across the model hierarchy.

Simulation model generation for aircraft electrical system

The experiment applies the proposed generative system modeling framework to the aircraft electrical system, which comprises six subsystems: power supply, flight scenario control module, control bus, radar, rudder, and thrust. Product design documents drawn from relevant literature and team-compiled modeling instructions served as inputs. Experiments were conducted on an A100 80 GB GPU platform, with CodeQwen1.5-7B fine-tuned using LoRA over 5,404 training samples across 3 epochs, achieving a final converged loss of 0.125. Eight graduate students familiar with X language participated as evaluators, following standardized training and independent scoring protocols.

Quantitative results demonstrate that the proposed method (Method A) substantially outperforms direct Transformer-based generation (Method B) across all three open-source models tested. Final scores under Method A reached 0.791, 0.812, and 0.845 for CodeGemma-7B, CodeQwen1.5-7B, and DeepSeek-Coder-6.7B, respectively, compared to scores of approximately 0.147–0.156 under Method B. The best open-source result (DeepSeek-Coder, Method A: 0.845) also surpassed the proprietary GPT-4o baseline (0.372), representing an overall quality improvement of approximately 21.4% for mainstream open-source Transformer-based models. ANOVA confirmed statistically significant differences among methods ($F(6, 42) > 1000$, $p < 0.001$, $\eta_p^2 = 0.999$), and inter-rater reliability was high (ICC(2,1) = 0.996 on complete data).

The NER-BERT model achieved 100% entity recall across all tested product design documents (aircraft electrical, electric vehicle, railroad crossing, and aircraft take-off systems), with entity precision ranging from 90.6% to 97.1%, confirming reliable extraction of system composition information. A notable qualitative finding was that Transformer-based models performed poorly on the Radar subsystem due to interference from prior domain knowledge conflicting with the design docu-

ment descriptions; renaming the module improved generation quality, highlighting a tension between model generalization and task-specific fidelity.

The proposed validation and simulation framework — comprising iterative compiler-guided syntax correction, human-in-the-loop refinement, and simulation-based verification — successfully transformed generated models into executable simulations. All subsystems were imported into the XLab development tool, compiled, and simulated, with results confirming correct state transitions and key performance indicators (e.g., current values) consistent with the product design document requirements.

Conclusion

The paper presents a method for generating simulation models of system physical properties using scalable templates and Transformer-based models within an MBSE framework. Key findings include: NER-BERT effectively filters redundant information from product design documents and reliably identifies system composition; the proposed evaluation metrics—incorporating Degree of Error and Model Consistency beyond standard code accuracy—enable quantitative assessment of simulation models; the scalable template-based generation approach overcomes LLM limitations with long-text inputs through modular code completion, improving simulation model quality by 21.4% over direct Transformer-based generation; and the validation and simulation framework efficiently transforms generated models into practically usable ones through iterative compiler feedback, human-in-the-loop correction, and simulation-based verification.

The authors identify the following directions for future work:

- Expanding the size and diversity of the Transformer-based model training dataset.
- Exploring the application of GenAI across other stages of product design beyond simulation model generation for physical properties.
- Improving NER-BERT accuracy (while maintaining full recall) through boundary refinement methods such as span-based tagging or CRF decoding, lightweight post-processing rules for irrelevant tokens, and domain-specific vocabularies.
- Extending the framework to additional application domains, such as electric vehicle systems and industrial robot control, to further demonstrate generalizability.

- Expanding the evaluator pool to improve the robustness and generalizability of the manual evaluation findings.
- Investigating how Transformer-based models can better balance the application of prior knowledge with task-specific requirements when generating simulation models from well-documented design documents.